Compte rendu TP-2

Introduction **4**
Mise en place du serveur web et hébergement des sites **4**
1 Configuration réseau 4
2 Création des arborescences des sites web 5
3 Création des VirtualHosts 5
4 Configuration des bases de données 5
5 Configuration du SFTP et des groupes 7
6 Gestion des droits ACL 8
7 Configuration du DNS 8
PARTIE 1 – Création de l'Autorité de Certification (CA) **11**
8.1 Création de l'arborescence pour la CA 11
8.2 Génération de la clé privée de la CA 13
8.3 Création du certificat auto-signé de la CA 13
PARTIE 2 – Création du certificat du serveur web **14**
9.1 Génération de la clé privée du serveur web pour chaque site 14
9.2 Génération des demandes de certificats (CSR) 15
9.3 Transfert vers ton serveur CA 16
9.4 Signature des certificats sur le serveur CA 17
9.5 Copier les certificats signés vers le serveur web 20
9.6 Configuration d'Apache pour chaque site HTTPS 20
9.7 Contenu et rôle de chaque partie du VirtualHost 443 20
9.8 Lien avec le VirtualHost sur le port 80 21
Conclusion **25**

Introduction

Dans ce TP, je devais mettre en place un serveur web complet et sécurisé sous Debian 12, capable d'héberger plusieurs sites.
Pour cela, j'ai installé et configuré différents services tels que Apache2, MariaDB, phpMyAdmin, SFTP, et un serveur DNS.
Deux sites ont été hébergés : gsb1 et gsb2, chacun ayant leur propre base de données, utilisateurs, et des restrictions d'accès via des groupes (**dev1** et **dev2**).
Enfin, j'ai mis en place un serveur de certificats (CA) afin de générer et signer les certificats SSL/TLS nécessaires pour sécuriser les deux sites en HTTPS.

Mise en place du serveur web et hébergement des sites

Sur mon serveur Debian 12, j'ai installé tous les modules nécessaires pour héberger mes deux sites web GSB1 et GSB2 :

apt-get update
apt-get upgrade

Ensuite j'ai installé :

- Apache2 (**apt install apache2**)
- Php (**apt install php-mysql**)
- SFTP (**apt install openssh-server**)
- PHPMYADMIN (**apt install phpmyadmin**)
- Maria DB (**apt install mariadb-server**)
- ACL (**apt install acl**) (Pour gérer les permissions sur les dossiers des sites)

1 Configuration réseau

Puis j'ai configuré une adresse IP static de mon interface grâce à la commande :
nano /etc/network/interfaces.d/

```
  GNU nano 7.2
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

source /etc/network/interfaces.d/*

# The loopback network interface
auto lo
iface lo inet loopback

# The primary network interface
allow-hotplug ens18
iface ens18 inet static

        address 172.16.152.10/16
        gateway 172.16.0.1
        dns_nameservers 172.16.0.100
```

2 Création des arborescences des sites web

J'ai ensuite créé deux sites web hébergés sur le même serveur :

```
mkdir -p /var/www/html/gsb1
mkdir -p /var/www/html/gsb2
```

Les fichiers de chaque site sont placés dans leur dossier respectif :

```
root@debian12:/var/www/html# ls -l
total 8
drwxrwx---+ 5 root dev1 4096 28 sept. 18:32 gsb1
drwxrwx---+ 5 root dev2 4096 28 sept. 18:23 gsb2
```

3 Création des VirtualHosts

Dans `/etc/apache2/sites-available/`, j'ai créé deux fichiers :

- `gsb1.conf`
- `gsb2.conf`

Dans lequel j'ai configuré le VirtualHost *:80 en mettant :

ServerName gsb1.152.sio
DocumentRoot /var/www/html/gsb1

Ensuite, j'ai activé les sites avec les commandes :

```
a2ensite gsb1.conf
a2ensite gsb2.conf
systemctl reload apache2
```

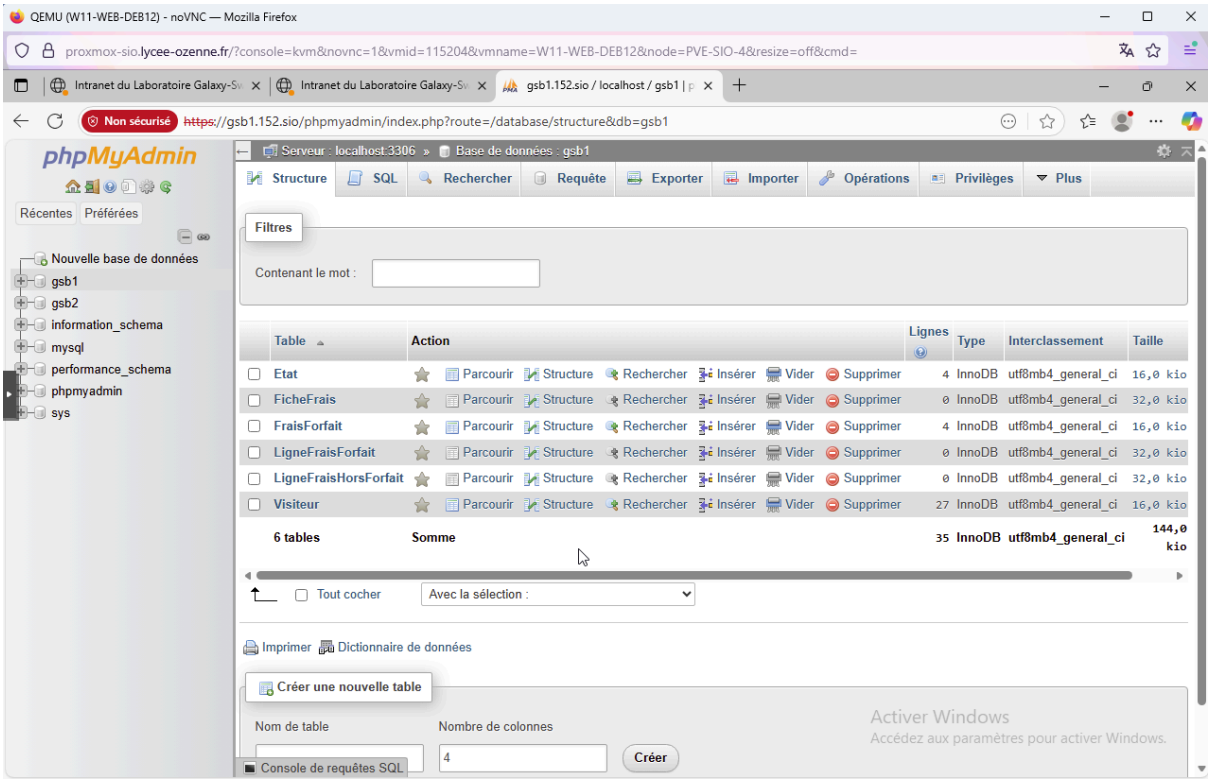
4 Configuration des bases de données

Ensuite je suis allé dans **/var/www/html/gsb1/include/_bdGestionDonnees.lib.php** :
où j'ai modifié les variables $login, $mdp et $bd :

```
172.16.152.10 - PuTTY
  GNU nano 7.2      _bdGestionDonnees.lib.php
 * @return resource identifiant de connexion
 */
function connecterServeurBD() {
    $hote = "localhost";
    $login = "admin";
    $mdp = "Azerty31";
    return mysqli_connect($hote, $login, $mdp);
}

^G Aide      ^O Écrire     ^W Chercher   ^K Couper
^X Quitter   ^R Lire fich. ^\ Remplacer  ^U Coller
```

Puis, j'ai créé deux bases et leurs utilisateurs dans MariaDB :

```
CREATE DATABASE gsb1;
CREATE DATABASE gsb2;

CREATE USER 'admin1'@'localhost' IDENTIFIED BY 'mdp1';
CREATE USER 'admin2'@'localhost' IDENTIFIED BY 'mdp2';
GRANT ALL PRIVILEGES ON gsb1.* TO 'admin1'@'localhost';
GRANT ALL PRIVILEGES ON gsb2.* TO 'admin2'@'localhost';

CREATE USER 'dev1'@'localhost' IDENTIFIED BY 'dev1';
CREATE USER 'dev2'@'localhost' IDENTIFIED BY 'dev2';
GRANT SELECT, INSERT, UPDATE, DELETE ON gsb1.* TO
'dev1'@'localhost';
GRANT SELECT, INSERT, UPDATE, DELETE ON gsb2.* TO
'dev2'@'localhost';
FLUSH PRIVILEGES;
```

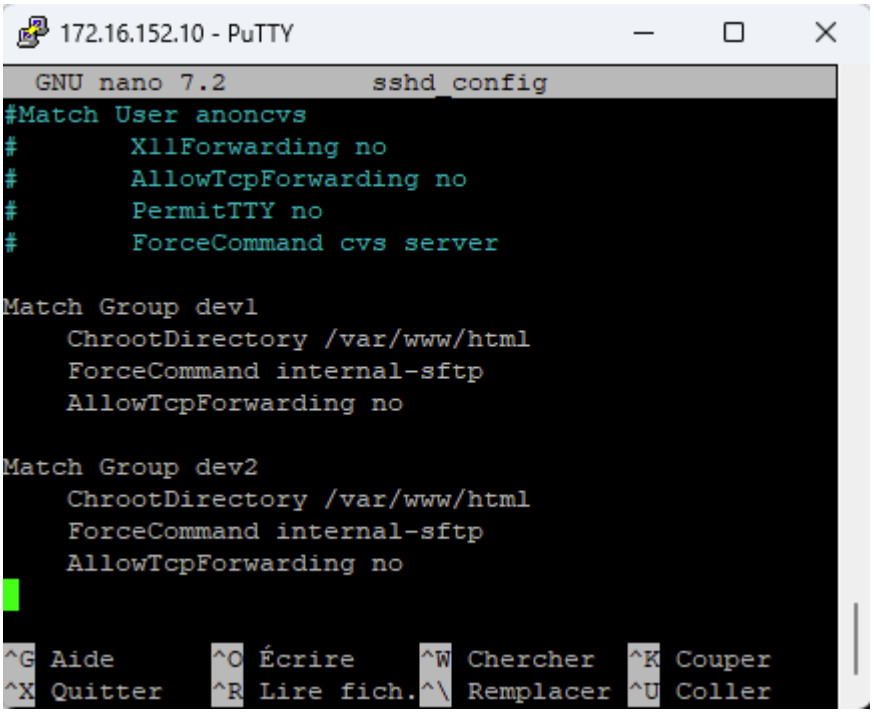


5 Configuration du SFTP et des groupes

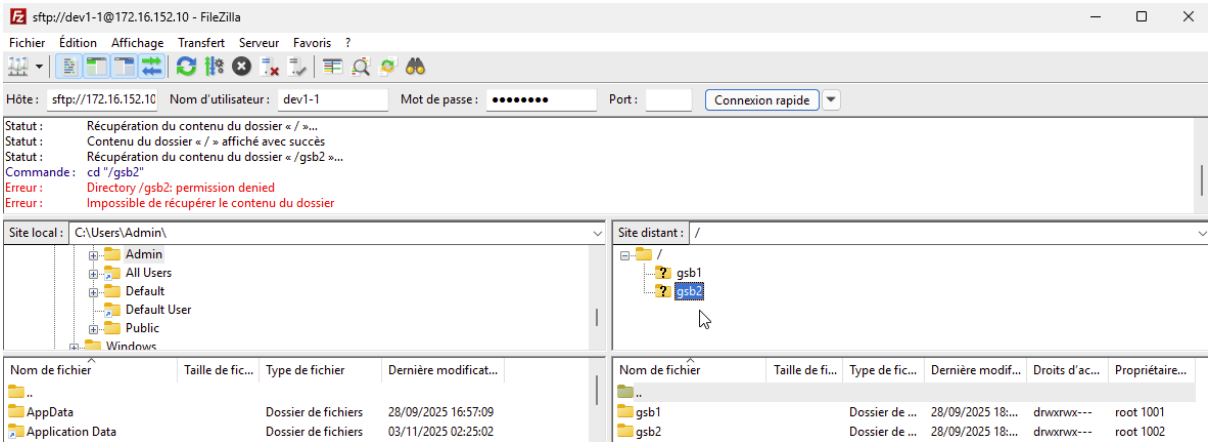
J'ai créé deux groupes de développement, chacun enfermé dans son répertoire ("chroot") :

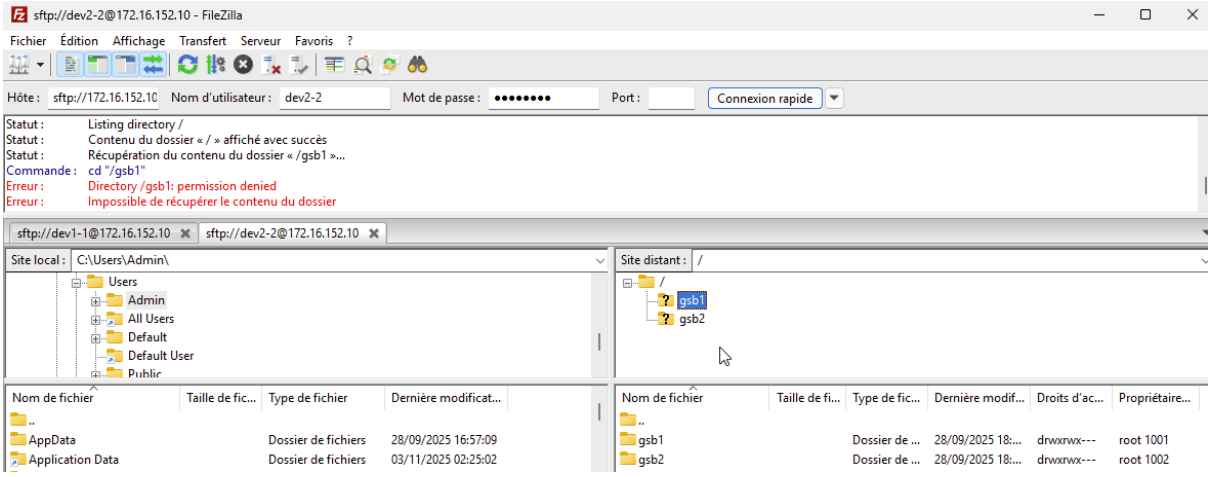
```
groupadd dev1
groupadd dev2

useradd -m -G dev1 dev1-1
useradd -m -G dev1 dev1-2
useradd -m -G dev2 dev2-1
useradd -m -G dev2 dev2-2
```

Dans **/etc/ssh/sshd_config**, j'ai ajouté :



On peut voir que les devs ont accés à leur site respectif et qu'ils ne peuvent pas remonter pour accéder à l'autre site (message de permission refusée) :





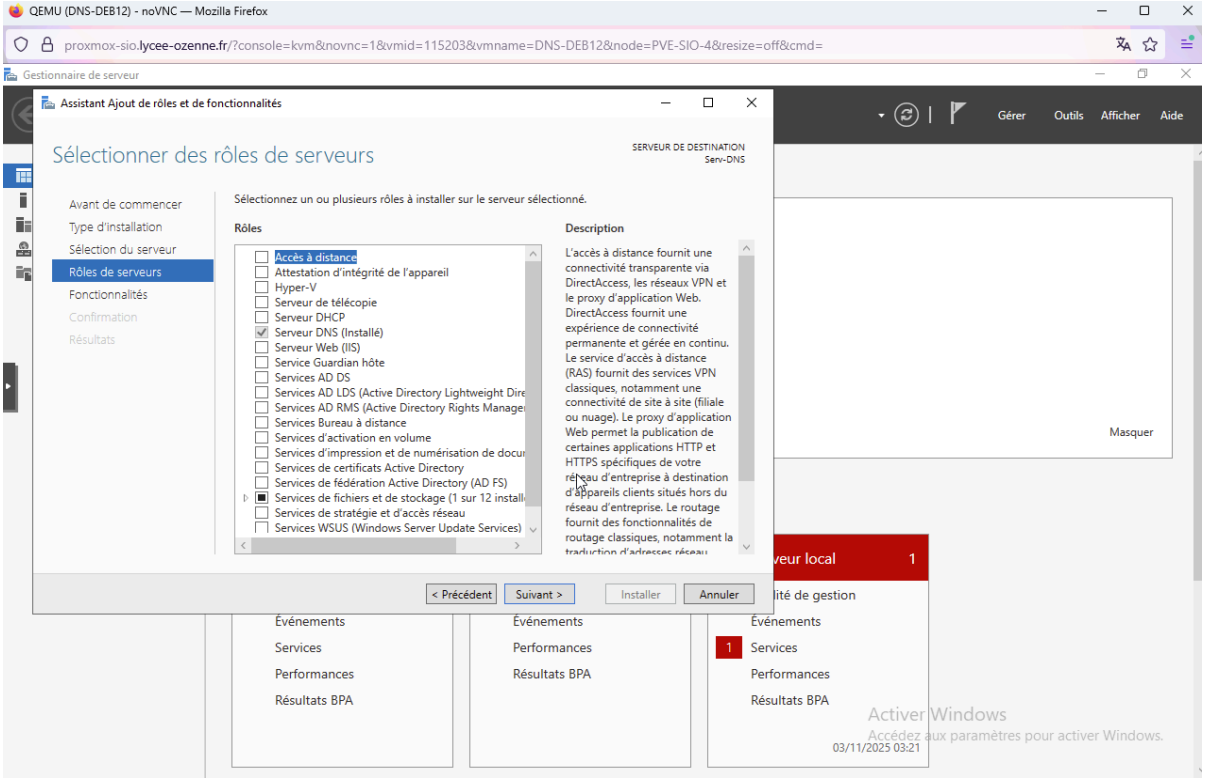
6 Gestion des droits ACL

Comme Apache (utilisateur `www-data`) doit pouvoir lire les fichiers des sites sans être dans les groupes `dev1` et `dev2`, j'ai utilisé les ACL :

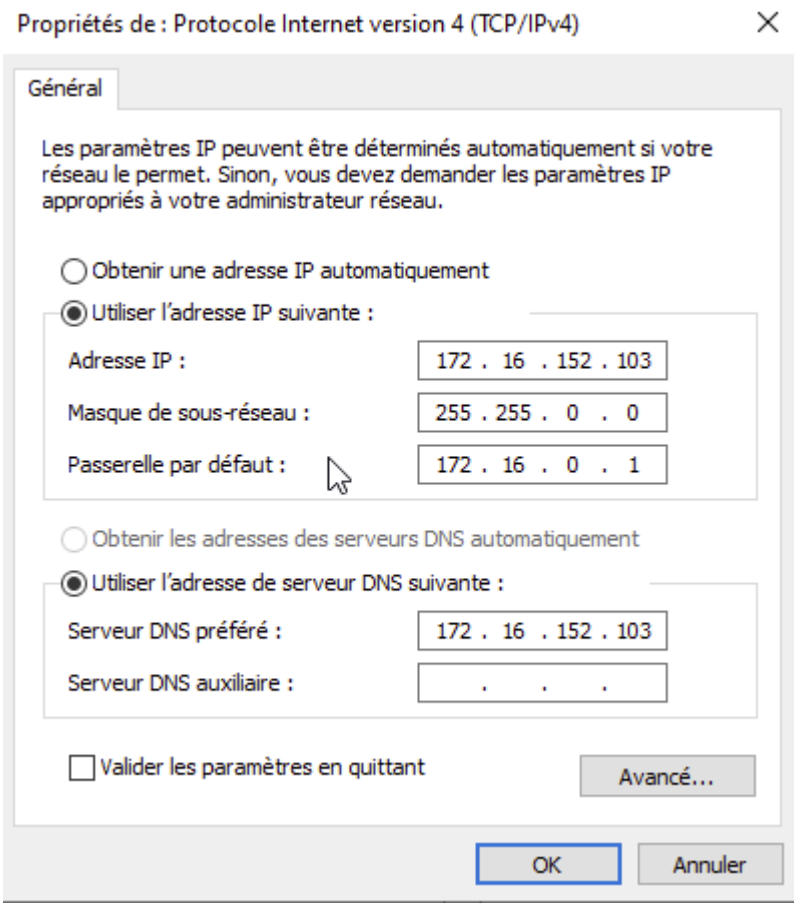
```
setfacl -R -m u:www-data:rx /var/www/html/gsb1
setfacl -R -m u:www-data:rx /var/www/html/gsb2
```

7 Configuration du DNS

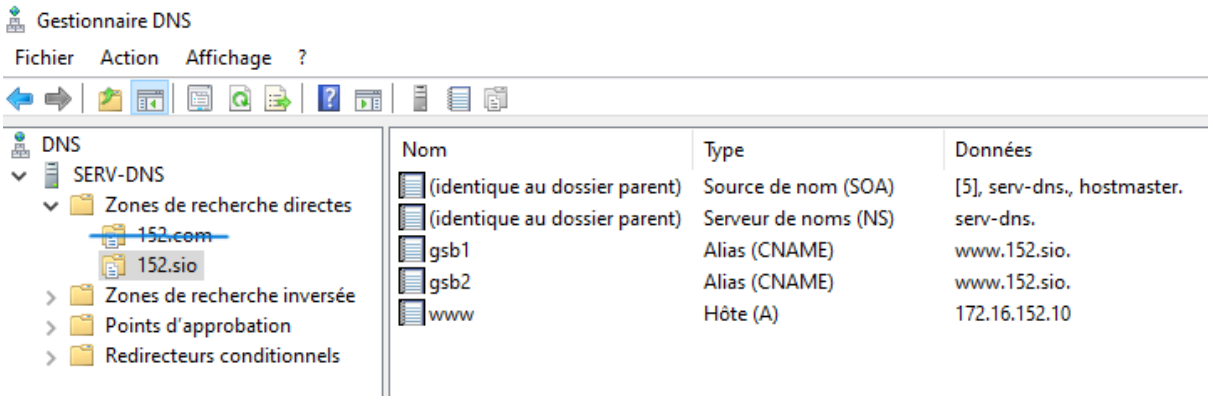
Après avoir créer ma VM pour mon DNS, j'ai fais Ajouter des Rôles et fonctionnalités et j'ai sélectionné serveur DNS :



Ensuite j'ai fais sa configuration IP :



Puis j'ai créé un nom de domaine (**152.sio**) avec un Hôte, et un Alias pour chacun des sites :



Création d'un Serveur de Certificat :

PARTIE 1 – Création de l'Autorité de Certification (CA)

Configuration du serveur :

apt-get update
apt-get upgrade

J'ai configuré l'interface du serveur de certificat en lui donnanr une adresse IP, une passerelle et mode statique grâce à la commande :
nano /etc/network/interfaces.d/

```
  GNU nano 7.2                                                    interfaces
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

source /etc/network/interfaces.d/*

# The loopback network interface
auto lo
iface lo inet loopback

# The primary network interface
allow-hotplug ens18
iface ens18 inet static

        address 172.16.152.11/16
        gateway 172.16.0.1
```

8.1 Création de l'arborescence pour la CA

Arborescence :

```
mkdir -p /etc/ssl/CA/{certs,crl,newcerts,private}
```

Explication :

mkdir = crée un dossier.
-p = crée aussi les sous-dossiers nécessaires s'ils n'existent pas.

On crée ici les répertoires utilisés par OpenSSL :

- **certs** → contiendra les certificats signés.
- **crl** → liste des certificats révoqués.
  newcerts → copie locale de tous les certificats créés.
  private → la clé privée de la CA (très sensible).

Droits :

Ensuite l'on donne tous les droits au proptiétaires seulement :

```
chmod 700 /etc/ssl/CA/private
```

Explication :

chmod 700 = seuls le propriétaire (root ici) peut lire, écrire, exécuter.
→ Sécurise le dossier des clés privées.

Création d'un fichier pour stocker les certificats :

```
touch /etc/ssl/CA/index.txt
```

Explication :

touch = crée un fichier vide.
index.txt sert de base de données pour enregistrer tous les certificats délivrés par la CA.

Initialise le prochain numéro de certificat :

```
echo 01 > /etc/ssl/CA/serial
```

```
root@debian12:~# echo 01 > /etc/ssl/CA/serial
```

Explication :

echo 01 → affiche le texte 01.

> → redirige ce texte vers un fichier au lieu de l'afficher à l'écran.

/etc/ssl/CA/serial → c'est le fichier dans lequel on écrit.

Cette commande crée (ou remplace) le fichier **`/etc/ssl/CA/serial`** et y écrit le nombre 01 (on met un 0 devant car c'est plus conforme vu que -x509 est en hexadécimal). Ce nombre sert de numéro de départ pour les certificats que ton autorité de certification va créer.

8.2 Génération de la clé privée de la CA

```
cd /etc/ssl/CA
openssl genrsa -des3 -out private/CAkey.pem 4096
```

Explication :

- `openssl genrsa` → crée une clé privée RSA.
- `-des3` → chiffre la clé avec un mot de passe (très important).
- `-out` → indique le nom du fichier de sortie.
- `4096` → longueur de clé en bits (plus c'est grand, plus c'est sécurisé).
  → Résultat : `/etc/ssl/CA/private/CAkey.pem` = ta **clé privée d'autorité**.

```
root@debian12:/etc/ssl/CA# openssl genrsa -des3 -out private/CAkey.pem 4096
Enter PEM pass phrase:
Verifying - Enter PEM pass phrase:
root@debian12:/etc/ssl/CA#
```

8.3 Création du certificat auto-signé de la CA

```
openssl req -new -x509 -days 1825 -key private/CAkey.pem -out
certs/CAcert.pem
```

Explication :

- `req` = création d'une requête de certificat.
- `-new` = nouvelle requête.
- `-x509` = auto-signé (la CA signe son propre certificat).
- `-days 1825` = durée de validité (ici 5 ans).
- `-key` = clé privée utilisée pour signer.
- `-out` = nom du certificat généré.
  → Résultat : `/etc/ssl/CA/certs/CAcert.pem` = certificat public de la CA.

Il faut ensuite donner des renseignements sur l'autorité de certification :

Country Name (2 letter code) [AU]:**FR**

State or Province Name (full name) [Some-State]:**Europe**

Locality Name (eg, city) : **Toulouse**

Organization Name (eg, company) [Internet Widgits Pty Ltd]: **GSB**

J'ai laissé le reste vide.

```
root@debian12:/etc/ssl/CA# openssl req -new -x509 -days 1825 -key private/CAkey.pem  -out certs/CAcert.pem
Enter pass phrase for private/CAkey.pem:
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:FR
State or Province Name (full name) [Some-State]:Europe
Locality Name (eg, city) []:Toulouse
Organization Name (eg, company) [Internet Widgits Pty Ltd]:GSB
Organizational Unit Name (eg, section) []:.
Common Name (e.g. server FQDN or YOUR name) []:.
Email Address []:.
root@debian12:/etc/ssl/CA#
```

PARTIE 2 – Création du certificat du serveur web

9.1 Génération de la clé privée du serveur web pour chaque site

On crée un dossier SSL pour chaque site du serveur web:

```
mkdir -p /etc/ssl/gsb1
mkdir -p /etc/ssl/gsb2
```

```
root@debian12:~# mkdir -p /etc/ssl/gsb1
root@debian12:~# mkdir -p /etc/ssl/gsb2
```

Puis on génère une clé privée pour chaque site.

gsb1:

```
openssl genrsa -out /etc/ssl/private/servGSB1key.pem 4096
```

gsb2 :

```
openssl genrsa -out /etc/ssl/private/servGSB2key.pem 4096
```

```
root@debian12:/etc/ssl/private# openssl genrsa -out /etc/ssl/private/servGSB1key.pem 4096
root@debian12:/etc/ssl/private# openssl genrsa -out /etc/ssl/private/servGSB2key.pem 4096
```

Explications :

- `openssl genrsa` → crée une clé RSA privée.
- `-out` → nom du fichier de sortie.
- `4096` → longueur de la clé (sécurité forte).
- **Pas de** `-des3` → sinon Apache demanderait le mot de passe à chaque démarrage.

Résultat :

```
root@debian12:/etc/ssl/private# ls -l
total 12
-rw------- 1 root root     3272  2 nov.  19:00 servGSB1key.pem
-rw------- 1 root root     3272  2 nov.  19:01 servGSB2key.pem
-rw-r----- 1 root ssl-cert 1704 15 sept. 14:28 ssl-cert-snakeoil.key
```

9.2 Génération des demandes de certificats (CSR)

gsb1 :

```
openssl req -new -key /etc/ssl/private/servGSB1key.pem -out
/etc/ssl/private/servGSB1dem.pem
```

```
Country Name (2 letter code) [AU]:FR
State or Province Name (full name) [Some-State]:.
Locality Name (eg, city) []:.
Organization Name (eg, company) [Internet Widgits Pty Ltd]:GSB
Organizational Unit Name (eg, section) []:dev1
Common Name (e.g. server FQDN or YOUR name) []:gsb1.152.sio
Email Address []:.

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:
An optional company name []:.
```

gsb2 :

```
openssl req -new -key /etc/ssl/private/servGSB2key.pem -out
/etc/ssl/private/servGSB2dem.pem
```

```
Country Name (2 letter code) [AU]:FR
State or Province Name (full name) [Some-State]:.
Locality Name (eg, city) []:.
Organization Name (eg, company) [Internet Widgits Pty Ltd]:GSB
Organizational Unit Name (eg, section) []:dev2
Common Name (e.g. server FQDN or YOUR name) []:gsb2.152.sio
Email Address []:.

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:
An optional company name []:.
```

Explications :

- **req -new → création d'une demande de certificat (CSR).**
- **-key → on utilise la clé privée du site.**
- **-out** → fichier de sortie contenant la demande.
- **CN** → nom DNS exact du site (ex. www.gsb.local)
  → Résultat : **/etc/ssl/GSB/servGSBdem.pem**

9.3 Transfert vers ton serveur CA

On copie les deux fichiers **.pem** de demande vers ton serveur CA depuis le serveur web :

scp /etc/ssl/private/servGSB1dem.pem root@172.16.152.11:/etc/ssl/CA/

```
root@debian12:~# scp /etc/ssl/private/servGSB1dem.pem root@172.16.152.11:/etc/ssl/CA/
```

```
The authenticity of host '172.16.152.11 (172.16.152.11)' can't be established.
ED25519 key fingerprint is SHA256:Qp2eyyWvKX01Xf8YV7VRJ0JmtJyKLuhhYfaiK0zmVkc.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '172.16.152.11' (ED25519) to the list of known hosts.
root@172.16.152.11's password:
```

```
root@172.16.152.11's password:
servGSB1dem.pem                                       100% 1647   835.4KB/s   00:00
```

scp /etc/ssl/private/servGSB2dem.pem root@172.16.152.11:/etc/ssl/CA/

```
root@debian12:~# scp /etc/ssl/private/servGSB2dem.pem root@172.16.152.11:/etc/ssl/CA/
root@172.16.152.11's password:
servGSB2dem.pem                                       100% 1647   808.5KB/s   00:00
```

Explications :

- **scp** = secure copy protocol, copie de fichiers via SSH.
- **root@172.16.152.11** → l'adresse IP de notre serveur CA.

9.4 Signature des certificats sur le serveur CA

(*Sur le serveur CA*)

nano /usr/lib/ssl/openssl.cnf

```
  GNU nano 7.2                    /usr/lib/ssl/openssl.cnf *
# OpenSSL may not work correctly which could lead to significant system
# problems including inability to remotely access the system.
# [default_sect]
# activate = 1


####################################################################
[ ca ]
default_ca      = CA_default            # The default ca section

####################################################################
[ CA_default ]

dir             = /etc/ssl/CA           # Where everything is kept
certs           = $dir/certs            # Where the issued certs are kept
crl_dir         = $dir/crl              # Where the issued crl are kept
database        = $dir/index.txt        # database index file.
#unique_subject = no                    # Set to 'no' to allow creation of
                                        # several certs with same subject.
new_certs_dir   = $dir/newcerts         # default place for new certs.

certificate     = $dir/cacert.pem       # The CA certificate
serial          = $dir/serial           # The current serial number
crlnumber       = $dir/crlnumber        # the current crl number

^G Aide      ^O Écrire      ^W Chercher    ^K Couper      ^T Exécuter    ^C Emplacement
^X Quitter   ^R Lire fich.  ^\ Remplacer   ^U Coller      ^J Justifier   ^/ Aller ligne
```

On remplace la ligne : “**dir = ./demoCA**” par “**dir = /etc/ssl/CA**” afin qu’au openssl cherche les clés et les certificats dans le dossier CA.

gsb1 :

```
cd /etc/ssl/CA
openssl ca -policy policy_anything -out certs/servGSB1cert.pem
-infiles servGSB1dem.pem
```

```
root@debian12:/etc/ssl/CA# openssl ca -policy policy_anything \
  -out certs/servGSB1cert.pem \
  -infiles servGSB1dem.pem
Using configuration from /usr/lib/ssl/openssl.cnf
Enter pass phrase for /etc/ssl/CA/private/cakey.pem:
Check that the request matches the signature
Signature ok
Certificate Details:
        Serial Number: 1 (0x1)
        Validity
            Not Before: Nov  2 20:29:44 2025 GMT
            Not After : Nov  2 20:29:44 2026 GMT
        Subject:
            countryName               = FR
            organizationName          = GSB
            organizationalUnitName    = dev1
            commonName                = gsb1.152.sio
        X509v3 extensions:
            X509v3 Basic Constraints:
                CA:FALSE
            X509v3 Subject Key Identifier:
                B3:2C:69:18:A6:22:32:CB:1B:3A:0E:57:53:32:90:BD:9F:59:EF:44
            X509v3 Authority Key Identifier:
                6F:C7:CF:6E:43:E4:F6:CC:08:8B:83:D8:CF:AD:F9:22:7B:C3:91:21
Certificate is to be certified until Nov  2 20:29:44 2026 GMT (365 days)
Sign the certificate? [y/n]:y


1 out of 1 certificate requests certified, commit? [y/n]y
Write out database with 1 new entries
Database updated
root@debian12:/etc/ssl/CA#
```

gsb2 :

```
openssl ca -policy policy_anything -out certs/servGSB2cert.pem
-infiles servGSB2dem.pem
```

```
root@debian12:/etc/ssl/CA# openssl ca -policy policy_anything -out certs/servGSB2cert.pem -infiles servGSB2dem.pem
Using configuration from /usr/lib/ssl/openssl.cnf
Enter pass phrase for /etc/ssl/CA/private/cakey.pem:
Check that the request matches the signature
Signature ok
Certificate Details:
        Serial Number: 2 (0x2)
        Validity
            Not Before: Nov  2 20:30:59 2025 GMT
            Not After : Nov  2 20:30:59 2026 GMT
        Subject:
            countryName               = FR
            organizationName          = GSB
            organizationalUnitName    = dev2
            commonName                = gsb2.152.sio
        X509v3 extensions:
            X509v3 Basic Constraints:
                CA:FALSE
            X509v3 Subject Key Identifier:
                E5:FE:C6:EB:CE:6F:D4:13:8F:BB:A1:92:3F:BF:2B:DE:3B:83:59:44
            X509v3 Authority Key Identifier:
                6F:C7:CF:6E:43:E4:F6:CC:08:8B:83:D8:CF:AD:F9:22:7B:C3:91:21
Certificate is to be certified until Nov  2 20:30:59 2026 GMT (365 days)
Sign the certificate? [y/n]:y


1 out of 1 certificate requests certified, commit? [y/n]y
Write out database with 1 new entries
Database updated
root@debian12:/etc/ssl/CA#
```

Explications :

- **openssl ca** → agit en tant qu'autorité de certification.
- **-policy policy_anything** → accepte tous les champs standards.
- **-out** → nom du certificat signé.
- **-infiles** → fichier de demande à signer.

Résultat :

```
root@debian12:/etc/ssl/CA/certs# ls -l
total 20
-rw-r--r-- 1 root root 1923  6 oct.  14:41 CAcert.pem
-rw-r--r-- 1 root root 6903  2 nov.  21:30 servGSB1cert.pem
-rw-r--r-- 1 root root 6903  2 nov.  21:31 servGSB2cert.pem
```

9.5 Copier les certificats signés vers le serveur web

Sur le serveur CA, on copie les certificats vers le serveur web :

```
scp /etc/ssl/CA/certs/servGSB1cert.pem
root@172.16.152.10:/etc/ssl/gsb1/

scp /etc/ssl/CA/certs/servGSB2cert.pem
root@172.16.152.10:/etc/ssl/gsb2/

scp /etc/ssl/CA/certs/CAcert.pem root@172.16.152.10:/etc/ssl/
```

```
root@debian12:~# scp /etc/ssl/CA/certs/servGSB1cert.pem root@172.16.152.10:/etc/ssl/gsb1/
The authenticity of host '172.16.152.10 (172.16.152.10)' can't be established.
ED25519 key fingerprint is SHA256:Qp2eyyWvKX01Xf8YV7VRJ0JmtJyKLuhhYfaiK0zmVkc.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '172.16.152.10' (ED25519) to the list of known hosts.
root@172.16.152.10's password:
servGSB1cert.pem                                              100% 6903     2.9MB/s   00:00
root@debian12:~# scp /etc/ssl/CA/certs/servGSB2cert.pem root@172.16.152.10:/etc/ssl/gsb2/
root@172.16.152.10's password:
servGSB2cert.pem                                              100% 6903     5.6MB/s   00:00
```

```
root@debian12:~# scp /etc/ssl/CA/certs/CAcert.pem root@172.16.152.10:/etc/ssl/
root@172.16.152.10's password:
CAcert.pem
```

Le fichier **CAcert.pem** est aussi nécessaire sur le serveur web pour que les navigateurs reconnaissent la chaîne de confiance.

9.6 Configuration d'Apache pour chaque site HTTPS

Sur notre serveur web :

On met www-data dans le groupe ssl-cert (groupe qui a accès à **/etc/ssl/private/**) :

```
sudo usermod -aG ssl-cert www-data
```

Puis on redémarre Apache : **systemctl restart apache2**

Ensuite, on active ssl avec la commande : **a2enmod ssl**

9.7 Contenu et rôle de chaque partie du VirtualHost 443

<VirtualHost *:443> : Indique qu'Apache doit appliquer cette configuration à toutes les adresses IP (*) sur le port 443 (HTTPS).

```
ServerName gsb1.152.sio
DocumentRoot /var/www/html/gsb1
```

`ServerName` : nom DNS du site (doit correspondre au **CN** du certificat).
`DocumentRoot` : dossier où se trouvent les fichiers du site (pages PHP, HTML, CSS...).

SSLEngine on : Active le module SSL/TLS d'Apache, obligatoire pour établir une connexion HTTPS.

```
SSLCertificateFile /etc/ssl/gsb1/servGSB1cert.pem
SSLCertificateKeyFile /etc/ssl/private/servGSB1key.pem
SSLCertificateChainFile /etc/ssl/CAcert.pem
```

- **SSLCertificateFile** → le certificat signé par la CA (prouve l'identité du site).
- **SSLCertificateKeyFile** → la clé privée du serveur (sert à chiffrer/déchiffrer les échanges).
- **SSLCertificateChainFile** → le certificat de la CA (chaîne de confiance).

Ces trois fichiers sont nécessaires pour établir une communication chiffrée entre le navigateur et le serveur.

```
<Directory /var/www/html/gsb1>
    Options Indexes FollowSymLinks
    AllowOverride All
    Require all granted
    DirectoryIndex cSeConnecter.php
</Directory>
```

- **Options Indexes FollowSymLinks** : autorise l'accès aux fichiers du dossier et le suivi des liens symboliques.
- **Require all granted** : autorise tous les clients à accéder au site.
- **DirectoryIndex cSeConnecter.php** : définit la page d'accueil par défaut (chargée automatiquement si on tape seulement l'URL du site).

9.8 Lien avec le VirtualHost sur le port 80

Le VirtualHost port 80 (HTTP) contient :

```
<VirtualHost *:80>
    ServerName gsb1.152.sio
    Redirect permanent / https://gsb1.152.sio/
</VirtualHost>
```

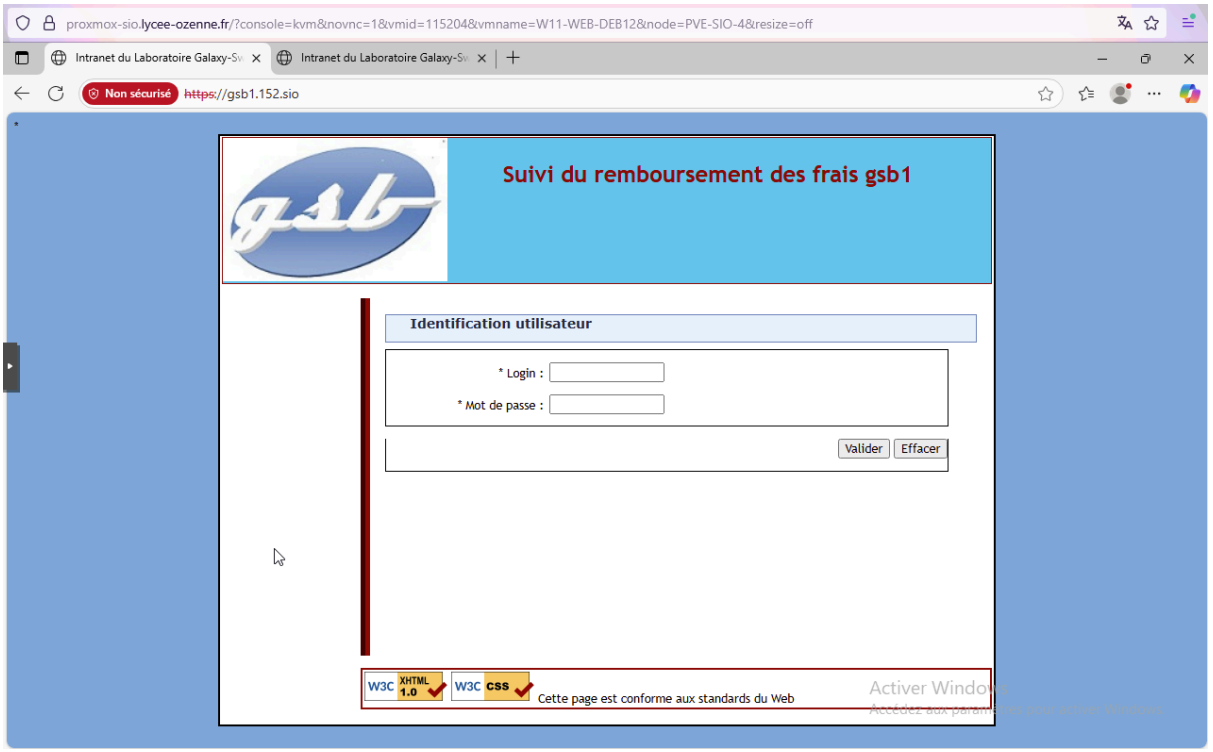
Ces 2 lignes servent uniquement à rediriger automatiquement les connexions non sécurisées (HTTP) vers la version HTTPS.
Ainsi, même si l'utilisateur tape **`http://gsb1.152.sio`**, il est redirigé vers **`https://gsb1.152.sio`**.

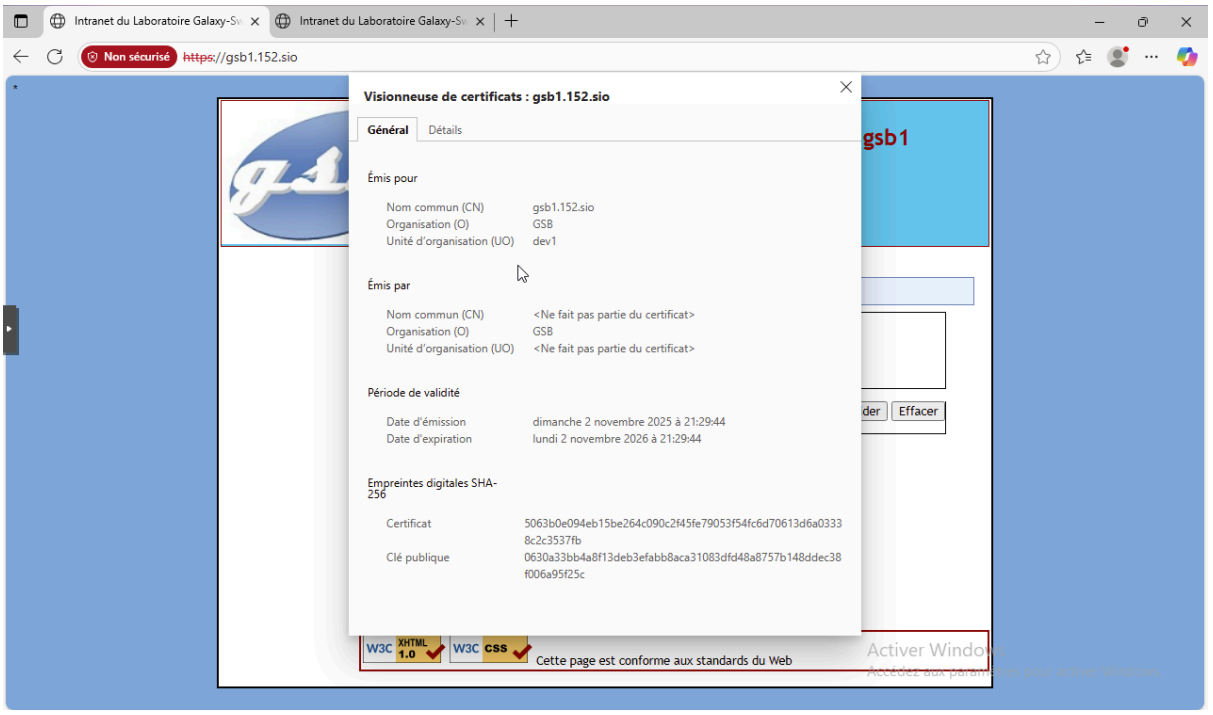
```
  GNU nano 7.2                                          gsb1.conf
<VirtualHost *:443>
        ServerName gsb1.152.sio
        ServerAlias www.152.sio
        DocumentRoot /var/www/html/gsb1

        SSLEngine on
        SSLCertificateFile /etc/ssl/gsb1/servGSB1cert.pem
        SSLCertificateKeyFile /etc/ssl/private/servGSB1key.pem
        SSLCertificateChainFile /etc/ssl/CAcert.pem

    <Directory /var/www/html/gsb1>
        Options Indexes FollowSymLinks
        AllowOverride All
        Require all granted
        DirectoryIndex cSeConnecter.php
    </Directory>

        ErrorLog ${APACHE_LOG_DIR}/gsb1_error.log
        CustomLog ${APACHE_LOG_DIR}/gsb1_access.log combined
</VirtualHost>

<VirtualHost *:80>
        ServerName gsb1.152.sio
        Redirect permanent / https://gsb1.152.sio/
</VirtualHost>
```

```
  GNU nano 7.2                                          gsb2.conf
<VirtualHost *:443>
        ServerName gsb2.152.sio
        ServerAlias www.152.sio
        DocumentRoot /var/www/html/gsb2

        SSLEngine on
        SSLCertificateFile /etc/ssl/gsb2/servGSB2cert.pem
        SSLCertificateKeyFile /etc/ssl/private/servGSB2key.pem
        SSLCertificateChainFile /etc/ssl/CAcert.pem

    <Directory /var/www/html/gsb2>
        Options Indexes FollowSymLinks
        AllowOverride All
        Require all granted
        DirectoryIndex cSeConnecter.php
    </Directory>

        ErrorLog ${APACHE_LOG_DIR}/gsb2_error.log
        CustomLog ${APACHE_LOG_DIR}/gsb2_access.log combined
</VirtualHost>


<VirtualHost *:80>
        ServerName gsb2.152.sio
        Redirect permanent / https://gsb2.152.sio/
</VirtualHost>
```

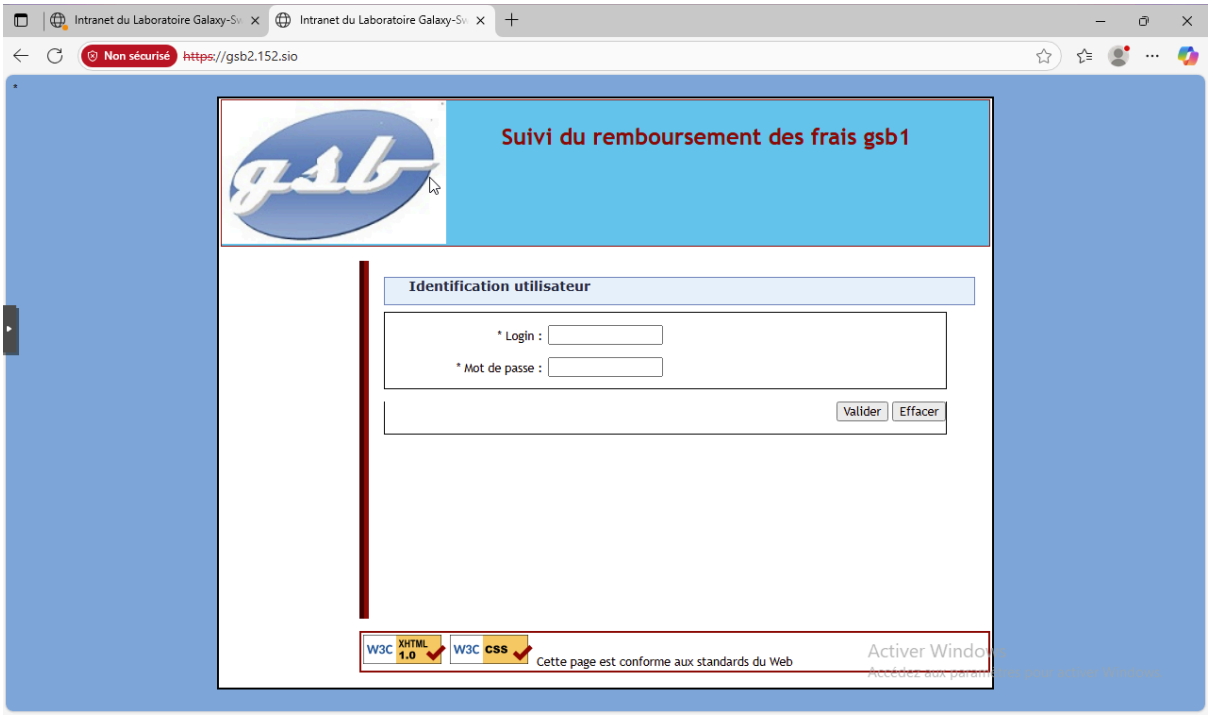
Les 2 sites sont donc bien accessibles en https.

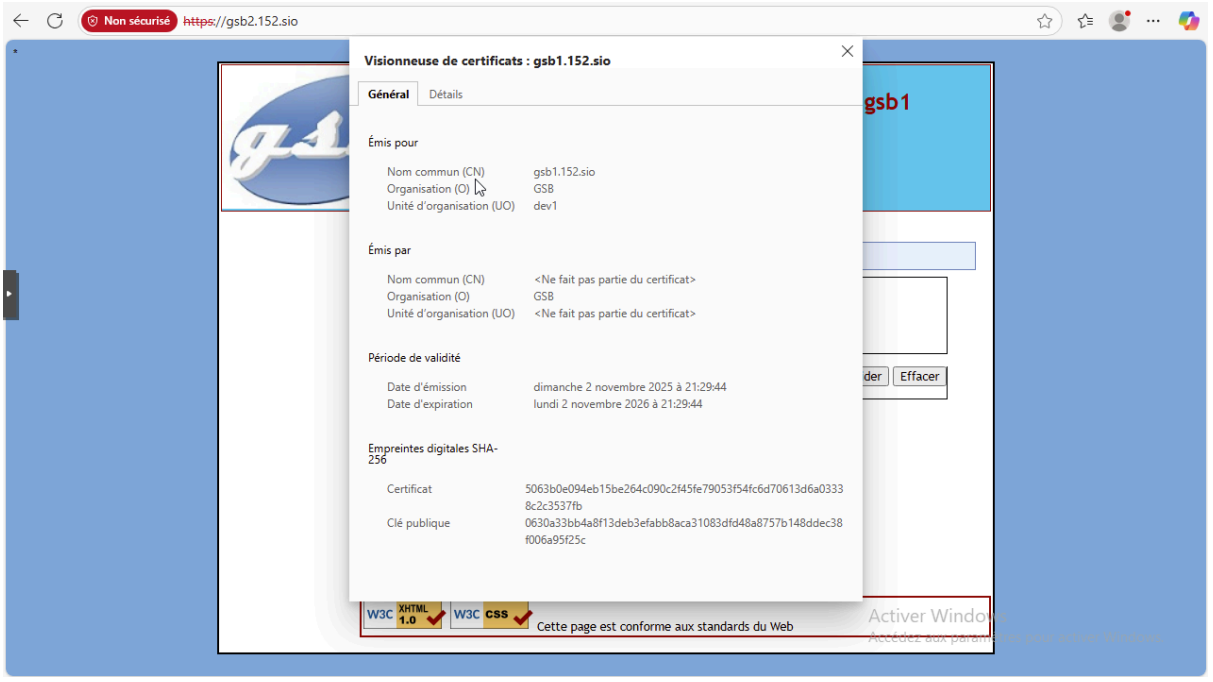
gsb1 (site1) :





gsb2 (site2):





Conclusion

A la fin de ce TP, les deux sites gsb1 et gsb2 sont accessibles et fonctionnels en HTTPS grâce au certificat signé par notre propre autorité de certification.
Les droits d'accès via SFTP permettent que chaque groupe de développeurs soit limité à son répertoire et ne puisse pas accéder à celui de l'autre.
Le DNS permet également d'accéder aux sites directement via leur nom de domaine.